

CO4 – AT4 - DATA INTERPRETATION

Question 1: Interpret the XML Structure

a. Identify the root element.

`<courses>`

The root element is `<courses>`. It is the outermost element and contains all the course records.

b. Identify the repeating record element.

`<course>`

The repeating record element is `<course>`. Each `<course>` element represents one course record. There are 5 course records.

c. Identify the attribute used to uniquely identify each course.

`id`

The `id` attribute uniquely identifies each course. Examples include C101, C102, C103, C104 and C105.

d. Identify the elements that represent numeric information.

`<students>`

`<credits>`

The `students` element represents the number of students enrolled, while the `credits` element represents the number of course credits.

e. State whether the XML document is structurally well-formed and justify your answer.

Yes, the XML document is well-formed. It has a single root element, every opening tag has a corresponding closing tag, elements are properly nested, the `id` attribute is enclosed in quotation marks, and the XML tags are correctly matched.

Question 2: Apply XPath for Data Selection

a. All course records.

`/courses/course`

b. Names of all courses.

`/courses/course/name`

c. Courses having more than 50 students.

`/courses/course[students > 50]`

d. Courses carrying 4 credits.

`/courses/course[credits = 4]`

e. Courses whose type is Theory.

`/courses/course[type = 'Theory']`

f. Names of Theory courses having more than 50 students.

`/courses/course[type = 'Theory' and students > 50]/name`

g. Faculty members handling courses with at least 4 credits.

`/courses/course[credits >= 4]/faculty`

h. The course whose id is C104.

`/courses/course[@id = 'C104']`

i. The first course available in the XML document.

`/courses/course[1]`

j. The last course available in the XML document.

`/courses/course[last()]`

Results for selected expressions:

More than 50 students: Web Technology (58), Artificial Intelligence (72), Machine Learning (64).

Exactly 4 credits: Web Technology, Artificial Intelligence, Machine Learning.

Theory courses: Web Technology, Artificial Intelligence, Machine Learning, Database Systems.

Theory courses with more than 50 students: Web Technology, Artificial Intelligence, Machine Learning.

Faculty handling courses with at least 4 credits: Dr. Arun, Dr. Meena, Dr. Priya.

C104: Machine Learning. First course: C101 – Web Technology. Last course: C105 – Database Systems.

Question 3: Apply XSLT for Data Presentation

The following XSLT transforms the XML data into an HTML table. It displays only courses having more than 40 students and sorts them in descending order of student enrollment.

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="html" indent="yes"/>
  <xsl:template match="/">
    <html>
      <head>
        <title>High Enrollment Courses</title>
      </head>
      <body>
        <h1>High Enrollment Courses</h1>
```

```

<table border="1">
  <tr>
    <th>Course Code</th>
    <th>Course Name</th>
    <th>Faculty</th>
    <th>Students</th>
    <th>Credits</th>
    <th>Type</th>
  </tr>
  <xsl:for-each select="courses/course[students > 40]">
    <xsl:sort select="students"
      data-type="number"
      order="descending"/>
    <tr>
      <td><xsl:value-of select="code"/></td>
      <td><xsl:value-of select="name"/></td>
      <td><xsl:value-of select="faculty"/></td>
      <td><xsl:value-of select="students"/></td>
      <td><xsl:value-of select="credits"/></td>
      <td><xsl:value-of select="type"/></td>
    </tr>
  </xsl:for-each>
</table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>

```

Important XPath condition used inside the XSLT:

`courses/course[students > 40]`

This condition ensures that only courses with more than 40 students are displayed.

Sorting condition:

```
<xsl:sort select="students"
          data-type="number"
          order="descending"/>
```

This sorts the selected courses according to student enrollment from highest to lowest.

Expected HTML output:

Course Code	Course Name	Faculty	Students	Credits	Type
AI302	Artificial Intelligence	Dr. Meena	72	4	Theory
ML304	Machine Learning	Dr. Priya	64	4	Theory
WEB301	Web Technology	Dr. Arun	58	4	Theory
DB305	Database Systems	Dr. Kumar	42	3	Theory

WEB303 – Web Technology Laboratory is not displayed because it has only 36 students, which is not greater than 40.

Question 4: Interpret the Extracted Data

a. Course with the highest enrollment:

Artificial Intelligence (AI302) has the highest enrollment with 72 students. It is handled by Dr. Meena and carries 4 credits.

b. Course with the lowest enrollment:

Web Technology Laboratory (WEB303) has the lowest enrollment with 36 students. It is handled by Dr. Ravi and carries 2 credits.

c. Number of Theory courses:

There are 4 Theory courses: Web Technology, Artificial Intelligence, Machine Learning and Database Systems.

d. Courses having exactly 4 credits:

Web Technology (WEB301), Artificial Intelligence (AI302), and Machine Learning (ML304) have exactly 4 credits. Therefore, there are 3 such courses.

e. Courses requiring additional teaching assistant support:

The condition is $\text{students} > 60$. Therefore, Artificial Intelligence (72 students) and Machine Learning (64 students) require additional teaching assistant support.

Final Summary

Requirement	Answer
Root element	<courses>
Repeating record	<course>
Unique attribute	id
Numeric elements	<students>, <credits>
Highest enrollment	Artificial Intelligence – 72
Lowest enrollment	Web Technology Laboratory – 36
Number of Theory courses	4
Courses with 4 credits	Web Technology, Artificial Intelligence, Machine Learning
Courses needing TA support	Artificial Intelligence, Machine Learning
XSLT filter	$\text{students} > 40$
XSLT sorting	Student enrollment – descending

Conclusion

The XML dataset provides structured information about university courses, including course codes, names, faculty, student enrollment, credits, and course type. XPath expressions can selectively retrieve course information based on conditions such as student count, credits and course type. XSLT can transform this XML data into an HTML table, filter courses based on enrollment, and sort them according to student strength. This enables the Academic Office to efficiently analyse course enrollment and prepare the semester workload report.